

Assignment 2 - Part 2 - COMP2152

Part 2 - Explain your Code (50%)

Write the answers in the doc provided and save and submit Part 2 as a PDF. You do not have to write or submit any new code for this section. I want you to understand how you could work on a piece of code that already exists (as is the case when working with Open Source code), and how to improve it. You can **type** in your answers *or* complete it **by hand** (handwriting MUST be legible) and then scan your submission.

1. How have we used classes for our project to reuse code?

We have used with object-oriented programming (OOP) for to foster code reusability via inheritance. The Character class acts as a parent class from which both Hero and Monster inherit. This averts duplication in code via the allowance for subclasses as well as sharing attributes like combat_strength and health_points, also methods, while they maintain behaviors special of their own.

2. Provide 1 line of code, as one of many examples, where code is shared between the monster class and the hero class?

A line of code that is responsible for initializing properties that both Hero and Monster share~~ class Character:

```
def __init__(self, name, health):  
    self.name = name  
    self.health = health
```

class Hero(Character):

```
def __init__(self, name, health, weapon):  
    super().__init__(name, health) # This line is shared  
    self.weapon = weapon
```

class Monster(Character):

```
def __init__(self, name, health, type_of_monster):  
    super().__init__(name, health) # This line is shared  
    self.type_of_monster = type_of_monster
```

In this example, the line `super().__init__(name, health)` is shared between the Hero and Monster classes. It calls the `__init__` method of the Character class, which is the parent class of both Hero and Monster. This line initializes the shared name and health attributes.

3. What is the benefit of using complex getters and setters?

Encapsulation: It carefully protects private attributes. It does so against direct modification.

Validation: Ensures legitimate values exclusively for attributes' assignment.

Flexibility: Permits specific alteration to basic reasoning without further adjustments to attribute access.

4. If we didn't use try-except blocks, what would be the problem?

Without try-except blocks, our program, in theory, could crash with invalid input from the user. As an example, the game would throw a `ValueError`. This would occur if a user enters certain strings instead of one number for dream levels. Exception handling vastly better program stability and user experience via crash prevention, also delivering error messages that fully guide users.

5. How could we use the name of the **operating system** or the **version of python** in your game to prevent errors? Choose just 1 of the above.

We can check the Python version to ensure compatibility. If an outdated version is detected, we can display a message prompting the user to update Python. For example:

```
python
CopyEdit
import sys
```

```
if sys.version_info < (3, 6):
    print ("Your Python version is outdated. Please update to Python 3.6 or later.")
    sys.exit(1) # Exit the game if the version is incompatible
```

6. What's another piece of information we could save inside of the save.txt file?
(Remember, we load this information every time we start a new game, so that we can keep track of all of the games you have played so far.)

We could store the highest level reached by the player to track progress across multiple game sessions. For example:

```
python
CopyEdit
# Saving the highest level
with open('save.txt', 'w') as file:
    file.write("Highest Level: 5") # Example: Player reached level 5
```

8. New Feature:

- a. Think of **1 new feature** you can add to the game that could use list comprehension and nested conditional statements. For now just write 1 sentence that describes the feature:

Now add your new feature description here:

Dynamic Enemy Spawning - Instead of a fixed number of monsters, the game will spawn a dynamic number of monsters based on the hero's level. This ensures difficulty scales with the player's experience.

Examples:

Below are the examples to show you that you can be very creative, and you should have fun with this exercise. You must use an idea that is NOT directly on the list below:

eg a) Add another monster so that the hero can fight 2 monsters at once

eg b) Create a digital board game, that shows the hero moving around to different towns on a map

eg c) Add a dog that runs in front of the hero and discovers features about the world

-
- b. Give the new feature you created a short 2-3 -word a title:

Now write your Title here:

Adaptive Battles

Examples:

eg a) Multiple Monsters

eg b) Roam Towns

eg c) Dog Scout

- c. Explain how you could implement the idea you chose. You must explain how you would use both of the control structures below. Draw a diagram, map, sketch for each (you can use any software for this, e.g. Draw.io). You don't have to match the style of diagram I have here, just use a visual to describe your idea. Note, you must have loops and conditional statements diagrammed below as needed:

Using a list comprehension loop

A list comprehension is a concise way to generate lists in Python. It allows you to create a new list by iterating over a range or collection, and it can also include conditions.

In this case, you want to dynamically create a list of monsters based on the hero's level. The higher the hero's level, the more monsters you generate. For example:

```
monsters = [Monster(level) for level in range(1, hero_level + 1)]
```

Using nested conditional statements

Once the monsters are generated, their attributes can be adjusted based on the hero's level. This ensures that monsters get progressively harder as the hero levels up.

A nested conditional statement (like an if-else structure inside another if else) can be used to dynamically adjust the monster attributes such as health, strength, and speed.

For example:

```

for monster in monsters:
    if hero_level < 5:
        monster.health = 50
        monster.attack = 10
    elif hero_level < 10:
        monster.health = 100
        monster.attack = 20
    else:
        monster.health = 150
        monster.attack = 30

```

Monster(level) represents creating a new monster based on the current level.

range(1, hero_level + 1) means that the loop will run from level 1 to the hero's current level.

The number of monsters will increase as the hero's level increases.

Example:

eg b) Roam Towns

i. Using a list comprehension loop

Every time in the loop, move one square in 1 direction, (N, E, S, W). Have a variable that keeps track of the Hero's location by saving values of the board. We can have 2 nested for loops and store the map as a 2D array.

Eg.

Hero location is currently at Row 3, Column D.

Town 2 location is at Row 4, Column G.

Town 1 location is at Row 1, Column A .

Diagram:

	A	B	C	D	E	F	G	H	I
1	(town1_loc)								
2									
3				(player_loc)					
4							(town2_loc)		

ii. Using a nested Conditional Statement

If the hero is in Town 2, then allow the hero to buy armor but not sell.

Otherwise, the hero can sell armor but cannot buy.

Create an array of armor options available in Town 2. He could also trade some of his loot based on the value of the loot he has.

